MÁSTER EN CIBERSEGURIDAD

MÓDULO 11. TRABAJO FIN DE MÁSTER (TFM)

La Crisis de la Gestión de Secretos: Secret Sprawl

Andrea Osma Rafael

Campus de Ciberseguridad

Resumen TFM del Máster de Ciberseguridad de la ENIIT sobre la problemática del Secret Sprawl en entornos DevSecOps. Se analiza la fuga de credenciales en repositorios y pipelines de CI/CD, proponiendo una arquitectura basada en bóvedas criptográficas y gestión dinámica de identidades.

Keywords: Ciberseguridad · DevSecOps · Secret Sprawl · Gestión de Secretos · Zero Trust

Índice general

MÁSTER EN CIBERSEGURIDAD MÓDULO 11. TRABAJO FIN DE MÁSTER (TFM) . . . . . .
Andrea Osma Rafael
1. Estado del Arte 3
1.1. Introducción: El Colapso del Perímetro Tradicional 3
1.1.1. El Paradigma Zero Trust y el Cumplimiento Normativo 3
1.2. La Crisis de la Gestión de Secretos: Secret Sprawl 3
1.2.1. Taxonomía de la Dispersión de Secretos 3
1.2.2. El Desafío de la Escala: Máquinas vs. Humanos 4
1.2.3. Riesgos de Seguridad Derivados del Secret Sprawl 4
1.2.4. El Ciclo de Vida del Secreto 5
1.2.5. Integración Transversal en el SDLC (DevSecOps) 5
1.3. Trazabilidad Forense y Dispositivos de Auditoría 6
2. Motivación 6
3. Hipótesis 7
4. Tesis: Arquitectura DevSecOps para la Gestión Dinámica de Secretos 8
4.1. Diseño de la Arquitectura Lógica 8
4.2. Resolución del Problema del "Secreto Cero" (Secret Zero) 8
4.3. Despliegue Inmutable y Operativa de Ingeniería 10
5. Demostración 10
5.1. Entorno de Pruebas 10
5.2. Estrategia de Despliegue mediante IaC 12
5.2.1. Instalación de dependencias en Proxmox 12
5.2.2. Aprovisionamiento Declarativo con Terraform 14
5.2.3. Configuración post-despliegue con Ansible 18
5.2.4. Anexo de código fuente del laboratorio 22
5.3. Metodología de evaluación 37
6. Resultados 37
6.1. Matriz de verificación operativa (evidencia directa) 37
6.2. Emisión dinámica de credenciales (TTL) y revocación 39
6.3. Trazabilidad de auditoría y detección 39
6.4. Reproducibilidad del despliegue mediante IaC 39

6.5. Pruebas complementarias con métricas y casos negativos 40
7. Discusión 40
7.1. Confirmación de hipótesis y relación con el Secret Sprawl 42
7.2. Amenazas a la validez y limitaciones 42
7.3. Trabajo futuro 42
8. Conclusiones 42
9. Glosario 43
10. Bibliografía 45

1. Estado del Arte

1.1. Introducción: El Colapso del Perímetro Tradicional

La evolución de la infraestructura IT, desde los centros de datos locales hacia la computación en la nube y las arquitecturas de microservicios, ha forzado una reevaluación total de los modelos de confianza. El modelo de seguridad perimetral, basado en la premisa de una red interna segura protegida por firewalls (Castillo y Foso), ha demostrado ser ineficaz en entornos donde las cargas de trabajo son efímeras y se distribuyen en múltiples nubes.

1.1.1. El Paradigma Zero Trust y el Cumplimiento Normativo Según la publicación especial NIST SP 800-207, la Arquitectura Zero Trust (ZTA) es un modelo de ciberseguridad que asume que no existe confianza implícita concedida a los activos o cuentas de usuario basándose únicamente en su ubicación física o de red. En lugar de defender un perímetro estático, la autenticación y autorización se evalúan de forma dinámica y continua por cada solicitud de acceso, protegiendo los recursos de manera individualizada (National Institute of Standards and Technology, 2020).

A nivel regulatorio, este cambio de paradigma se ha materializado en marcos europeos recientes como la directiva NIS2 y DORA (Digital Operational Resilience Act). Estas normativas exigen a las organizaciones, especialmente en infraestructuras críticas y el sector financiero, implementar controles estrictos y demostrables sobre la gestión de identidades, la resiliencia operativa y el riesgo de la cadena de suministro. La rotación automatizada de credenciales y la auditoría continua pasan de ser buenas prácticas a ser obligaciones legales estrictas (European Parliament and the Council of the European Union, 2022a, 2022b).

En el paradigma actual, la red es dinámica y, por definición, hostil. Por tanto, la seguridad debe desplazarse desde la red hacia la identidad. Este cambio sitúa a la Gestión de Identidad y Acceso (IAM) no como un servicio de soporte, sino como el plano de control central de la seguridad de la información. Este TFM analiza soluciones diseñadas para mitigar los riesgos derivados de la proliferación de identidades no humanas.

1.2. La Crisis de la Gestión de Secretos: Secret Sprawl

1.2.1. Taxonomía de la Dispersión de Secretos El término secret sprawl define la distribución descontrolada de credenciales (claves API, certificados, tokens, contraseñas) a lo largo del ciclo de vida del desarrollo de software (SDLC). En un entorno de Infraestructura como Código (IaC), el riesgo aumenta considerablemente.

- **Fugas en Control de Versiones:** La inmutabilidad de los sistemas como Git implica que un secreto subido accidentalmente persiste en el historial, requiriendo un proceso complejo de rewriting history o la invalidación inmediata de la credencial.

- **Variables de Entorno Inseguras:** El uso de archivos .env o variables de sistema sin cifrar en servidores de CI/CD (Jenkins, GitHub Actions) expone credenciales a cualquier proceso con privilegios de lectura en el entorno.
- **Certificados X.509 y Claves SSH Compartidas:** La emisión de certificados de larga duración o el uso de pares de claves SSH estáticas sin el respaldo de una Autoridad de Certificación (CA) efímera provoca que, si el material privado es comprometido, el acceso se mantenga de forma prolongada sin ser detectado.
- **Credenciales Hardcodeadas en Código Fuente:** La práctica de incrustar tokens o contraseñas directamente en los repositorios expone la información a cualquier usuario con privilegios de lectura y propaga el secreto a través de los artefactos de construcción.

1.2.2. El Desafío de la Escala: Máquinas vs. Humanos Investigaciones recientes sugieren que las identidades de máquinas (servicios, contenedores, bots) crecen a un ritmo cinco veces superior al de las identidades humanas. A diferencia de un operador humano, que puede validar su identidad aportando biometría o un token de hardware en un flujo MFA[1] tradicional, una carga de trabajo automatizada carece de este factor interactivo ("algo que eres" o "algo que tienes").

Esto exige sustituir el MFA[2] humano por la validación concurrente de múltiples atributos criptográficos y contextuales. Se requiere la presentación de credenciales como certificados TLS[3] mutuos (mTLS[4]) o tokens JWT firmados, combinados con la verificación de la IP de origen, el namespace o la rama del repositorio en tiempo de ejecución. Se espera que la validación estricta de estos metadatos limite la emisión de secretos exclusivamente a las cargas de trabajo legítimas (Jones & Bradley, 2015).

1.2.3. Riesgos de Seguridad Derivados del Secret Sprawl El almacenamiento incontrolado y la exposición de credenciales generan vectores de ataque críticos en las infraestructuras modernas. La literatura académica señala que la exposición de secretos en pipelines de CI/CD y en repositorios de código facilita la ejecución de los siguientes escenarios de compromiso:

- **Movimiento Lateral y Escalada de Privilegios:** En arquitecturas planas o basadas en microservicios, un atacante que obtiene un token de acceso básico puede pivotar hacia sistemas críticos o bases de datos si no existe una autenticación estricta.
- **Ataques a la Cadena de Suministro (Supply Chain Attacks):** La inyección de código malicioso en los flujos de integración continua a menudo se logra explotando credenciales expuestas en los repositorios, lo que compromete las fases de construcción y despliegue del software de forma automatizada.
- **Persistencia a Largo Plazo:** Las credenciales estáticas que carecen de rotación permiten a los atacantes mantener un acceso indetectable a la infraestructura (comportamiento esperado tras

una intrusión exitosa), incluso después de que los equipos de respuesta a incidentes mitiguen la vulnerabilidad técnica inicial que permitió la entrada.

1.2.4. El Ciclo de Vida del Secreto Para mitigar los riesgos asociados a la dispersión de credenciales, la gestión de secretos debe tratarse como un proceso continuo e integrado. Los secretos se consideran artefactos de primer nivel que requieren un ciclo de vida definido desde su creación hasta su destrucción. Este ciclo consta de las siguientes fases:

1. **Generación:** Creación de la credencial, preferiblemente de forma dinámica y asociando un Tiempo de Vida (TTL) estrictamente limitado a la misma.
2. **Almacenamiento Seguro:** Custodia del secreto en un almacén centralizado que proporcione cifrado y aplique controles de acceso basados en políticas (Policy-as-Code[5]) para definir quién, y bajo qué condiciones, puede interactuar con el dato.
3. **Distribución e Inyección:** Entrega temporal del secreto a la identidad que lo solicita (humana o de máquina) en tiempo de ejecución, evitando en todo momento su escritura en disco.
4. **Rotación:** Renovación periódica o basada en eventos de las credenciales, un proceso fundamental para invalidar versiones anteriores y acotar drásticamente la ventana de exposición ante una posible brecha.
5. **Revocación y Auditoría:** Invalidación del secreto en el sistema de destino tras expirar su TTL, manteniendo un registro inmutable de todas las operaciones para facilitar labores de cumplimiento normativo y diagnóstico forense.

1.2.5. Integración Transversal en el SDLC (DevSecOps) La gestión de identidades y secretos interactúa a lo largo de todo el Ciclo de Vida del Desarrollo de Software (SDLC). Aplicar el principio de shift-left security exige implementar controles técnicos desde las fases más tempranas:

- **Fase de Código (Code and Commit):** Constituye la primera línea de defensa. Dado que es habitual que se introduzcan credenciales hardcodeadas de manera inadvertida, en esta etapa se despliegan escaneos de secretos y hooks de pre-commit en los repositorios locales para bloquear la inyección en el control de versiones (GitHub, s.f.; OWASP, s.f.).
- **Fase de Construcción (Build and CI):** Los servidores de Integración Continua requieren acceso a repositorios de artefactos y entornos de prueba. El uso de variables de entorno estáticas en esta fase representa un riesgo significativo de fuga de datos. Los secretos deben ser inyectados dinámicamente y enmascarados para evitar su volcado accidental en los registros (logs) de ejecución.
- **Fase de Pruebas (Test):** Las herramientas de análisis dinámico e interactivo (DAST[6] e IAST[7]) necesitan credenciales temporales para ejecutar sus escaneos en los entornos de validación. Esta fase garantiza la separación estricta entre las credenciales de prueba y las de producción.

- **Fase de Despliegue y Operación (Deploy and Run):** En la etapa final, las cargas de trabajo contenerizadas u orquestadas solicitan sus credenciales en tiempo de ejecución. El gestor central valida la identidad criptográfica de la carga de trabajo y entrega un acceso efímero vinculado a políticas contextuales.

1.3. Trazabilidad Forense y Dispositivos de Auditoría

Un pilar fundamental de la Arquitectura Zero Trust es la visibilidad total sobre las peticiones de acceso. Durante las pruebas, se habilitó el motor de auditoría de Vault[8] para registrar cada interacción en formato JSON. Este registro inmutable captura el identificador de la entidad solicitante, la política evaluada, la ruta del secreto y la marca de tiempo exacta (HashiCorp, s.f.-a).

La exportación de estos registros estructurados hacia un sistema SIEM centralizado permite la detección de anomalías en tiempo real, como múltiples intentos de acceso denegados desde una IP específica, garantizando así los requisitos de resiliencia y auditoría continua exigidos por marcos normativos actuales.

2. Motivación

La transición de arquitecturas monolíticas a ecosistemas distribuidos, contenerizados y orientados a microservicios ha redefinido radicalmente el perímetro de seguridad clásico. En este paradigma actual, la red subyacente se asume como un entorno intrínsecamente hostil, desplazando a la identidad hacia el centro del plano de control. Este avance arquitectónico, si bien ha dotado de agilidad al desarrollo, ha precipitado una crisis operativa sin precedentes: la proliferación descontrolada de identidades no humanas, tales como servicios, contenedores y automatizaciones de despliegue. A diferencia de los usuarios humanos, estas entidades requieren mecanismos de autenticación criptográfica automatizados y de alta frecuencia, lo que ha derivado en el fenómeno conocido como Secret Sprawl (dispersión de secretos) a lo largo del ciclo de vida del desarrollo de software (SDLC).

En paralelo, la consolidación de las metodologías ágiles ha convertido a los repositorios de código y a las plataformas de Integración y Despliegue Continuo (CI/CD) en la columna vertebral de la ingeniería de software y, consecuentemente, en vectores de ataque de alto valor. La persistencia técnica de incrustar credenciales estáticas (hardcoding) o de utilizar variables de entorno debilmente protegidas expone a las infraestructuras a riesgos severos, facilitando el movimiento lateral y el compromiso de la cadena de suministro. Este escenario se vuelve críticamente complejo en sectores sometidos a estrictas normativas (como defensa, banca o infraestructuras críticas), donde la soberanía del dato exige despliegues en nubes privadas (on-premise) fuertemente aisladas. Históricamente, la innovación en la inyección dinámica de credenciales se ha diseñado de forma nativa para la nube pública, dejando a las infraestructuras locales relegadas a prácticas heredadas y a un control perimetral obsoleto.

La gestión manual de material criptográfico, como la rotación de contraseñas o la renovación de certificados X.509, no solo introduce una carga operativa severa para los equipos de ingeniería, sino que también incrementa la probabilidad de interrupciones del servicio debido a errores humanos u omisiones.

La motivación fundamental de este proyecto es cerrar esta brecha tecnológica, demostrando la viabilidad de integrar los más altos estándares de seguridad y gestión de identidad efímera en entornos privados restrictivos. Se busca evidenciar que la adopción de una cultura DevSecOps genuina donde la seguridad se codifica e inyecta en las fases iniciales del desarrollo (shift-left) permite sustituir los secretos estáticos por flujos de autenticación dinámicos bajo demanda. Al abstraer toda esta arquitectura mediante herramientas de Infraestructura como Código (IaC), se persigue establecer un modelo operativo inmutable y auditable desde la línea de comandos. Esta aproximación técnica pretende sentar las bases para una migración interoperable hacia arquitecturas de nube híbrida, estandarizando los controles de acceso independientemente del hipervisor o proveedor subyacente.

3. Hipótesis

La presente investigación plantea que la integración de una arquitectura de seguridad centrada en la identidad, materializada a través de una bóveda criptográfica centralizada, reduce sustancialmente la superficie de ataque en entornos de nube privada sujetos a estrictos requisitos de soberanía del dato y aislamiento. Al transicionar de un modelo de confianza perimetral estático a un enfoque anclado en los principios de Zero Trust, donde la inyección de credenciales se realiza de forma dinámica y estrictamente bajo demanda, se espera que la ventana de oportunidad para la explotación de credenciales comprometidas se acote drásticamente frente a ataques automatizados.

Asimismo, se hipotetiza que la inserción de controles de seguridad de forma transversal (SAST[9], DAST[10] y análisis de composición de software) directamente en el pipeline de Integración y Despliegue Continuo (CI/CD) madura la postura de seguridad del ciclo de vida del software. Retirar el material criptográfico persistente del código fuente y de los repositorios de control de versiones mitiga el riesgo de Secret Sprawl y dificulta el movimiento lateral o la escalada de privilegios en caso de una intrusión inicial.

Finalmente, desde el punto de vista de la ingeniería de la infraestructura, se postula que la orquestación de este ecosistema mediante paradigmas de Infraestructura como Código (IaC) desvinculando la capa de gestión de acceso de la topología de red subyacente proporciona la abstracción necesaria para escalar el modelo operativo. El uso de manifiestos declarativos y automatización modular facilita la exportación de esta misma arquitectura desde un entorno on-premise virtualizado hacia un modelo de nube híbrida pública, manteniendo la cohesión y las políticas de seguridad inmutables sin requerir una reingeniería profunda de la lógica de acceso.

4. Tesis: Arquitectura DevSecOps para la Gestión Dinámica de Secretos

La propuesta central de este proyecto consiste en el diseño e implementación de una arquitectura centralizada de gestión de secretos, fundamentada en el paradigma Zero Trust. El objetivo principal es la sustitución de credenciales estáticas por flujos de inyección dinámica, orquestados íntegramente mediante Infraestructura como Código (IaC).

4.1. Diseño de la Arquitectura Lógica

El núcleo del sistema reside en una bóveda criptográfica de alta disponibilidad (como HashiCorp Vault[11]), configurada para operar como la única fuente de verdad para el material sensible. La arquitectura se segmenta en tres planos funcionales:

- **Plano de Autenticación:** Las cargas de trabajo (contenedores, pipelines de despliegue) no poseen secretos iniciales estáticos en disco. Se autentican contra la bóveda utilizando identidades nativas de su entorno de ejecución, tales como tokens JWT en entornos de CI/CD mediante OIDC[12] (OpenID Connect) o Service Accounts en orquestadores (Jones & Bradley, 2015; OpenID Foundation, 2014).
- **Plano de Autorización:** Una vez autenticada la entidad no humana, un motor de políticas (Policy-as-Code[13]) evalúa de forma estricta si la identidad posee los privilegios mínimos necesarios para acceder a la ruta del secreto solicitado.
- **Plano de Emisión Dinámica:** En lugar de devolver una credencial de larga duración, el sistema genera credenciales efímeras bajo demanda (por ejemplo, usuarios temporales de bases de datos o certificados de corta duración) con un Tiempo de Vida (TTL) ajustado al tiempo de ejecución de la tarea.

4.2. Resolución del Problema del "Secreto Cero" (Secret Zero)

El desafío técnico más complejo en la automatización de secretos es el método para entregar el primer vector de autenticación a una máquina sin incurrir en prácticas de hardcoding. La solución propuesta aborda este dilema delegando la raíz de confianza en el proveedor de infraestructura subyacente.

Al emplear federación de identidades (OIDC[14]) para los flujos de integración continua, el pipeline presenta a la bóveda un token criptográfico firmado temporalmente por el proveedor de CI/CD. La bóveda verifica la firma pública de este token frente a las claves públicas del proveedor. En consecuencia, el código fuente y los manifiestos de despliegue permanecen completamente limpios de material criptográfico persistente. Se espera que este diseño acote significativamente la superficie de exposición en los repositorios de código, si bien la integridad total depende de la robustez del proveedor de identidades externo, constituyendo un comportamiento esperado de mitigación, no una protección absoluta ante vulnerabilidades de terceros (OpenID Foundation, 2014).

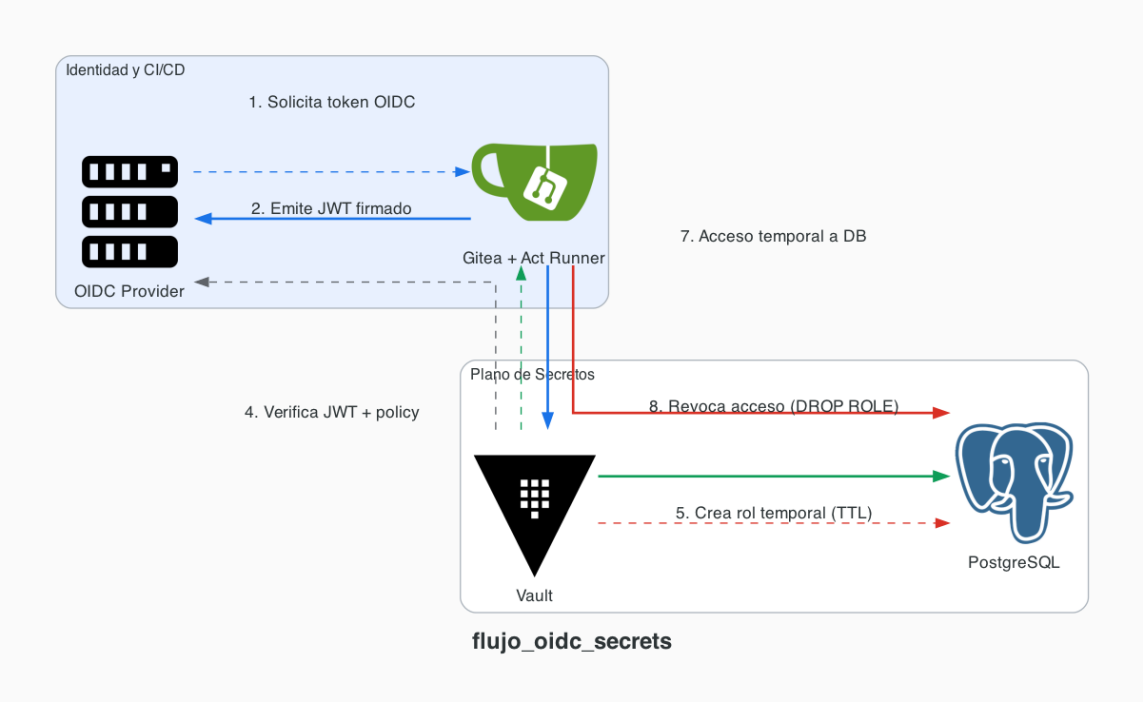

Figura 1. Secuencia de autenticación federada mediante OIDC y emisión/revocación de credenciales efímeras en Vault para acceso temporal a PostgreSQL.

4.3. Despliegue Inmutable y Operativa de Ingeniería

La abstracción de toda la infraestructura y configuración de seguridad se define mediante manifiestos declarativos. La inicialización de la bóveda, la definición de roles y la asignación de políticas de acceso se gestionan como código, permitiendo un versionado estricto y un control de cambios mediante flujos de revisión.

Fomentando una operativa técnica de alto rendimiento y reduciendo la fricción en la automatización, la interacción con el sistema en fases de administración e ingeniería se realiza exclusivamente mediante herramientas de interfaz de línea de comandos (CLI[15]) y editores de terminal. Se descarta la dependencia de interfaces gráficas (GUI), lo que facilita la creación de scripts de automatización y establece un registro de comandos auditable para cada modificación en el estado de la seguridad. Es por este motivo que la arquitectura propuesta se despliega mediante Terraform (HashiCorp, s.f.-c).

5. Demostración

5.1. Entorno de Pruebas

Para la validación empírica de la arquitectura propuesta, se ha desplegado un laboratorio *on-premise* sobre un hipervisor tipo 1 (Proxmox VE 9). La infraestructura de hardware asignada consta de 48 GB de memoria RAM, un disco de estado sólido (SSD) de 256 GB para el almacenamiento operativo de alta velocidad gestionado mediante aprovisionamiento fino (*thin provisioning*), y un disco duro mecánico (HDD) de 1 TB destinado a volcados de seguridad periódicos (`vzdump`).

La topología de red y computación se segmenta utilizando una combinación de contenedores Linux (LXC[16]) para servicios base y Máquinas Virtuales (VM) para la orquestación, distribuidos de la siguiente manera:

- **LXC[17] 1 (Gitea v1.21 + Act Runner):** Plataforma ligera para el control de versiones y la ejecución de pipelines de CI/CD, configurada para emitir tokens OIDC[18].
- **LXC[19] 2 (HashiCorp Vault[20] v1.15):** Bóveda criptográfica central con motor de almacenamiento integrado (Raft[21]) en configuración de nodo único.
- **LXC[22] 3 (PostgreSQL v16):** Motor de base de datos relacional que actúa como sistema objetivo para la inyección dinámica de credenciales.
- **VM 1 (k3s[23] v1.28):** Clúster de Kubernetes[24] ligero sobre una distribución Debian minimalista. El uso de una máquina virtual proporciona aislamiento a nivel de kernel, lo cual mitiga vectores de escape de contenedores (Kubernetes Authors, s.f.; Rancher Labs, s.f.).

Fomentando una operativa técnica ágil, la administración del entorno y la configuración de políticas en Vault[25] se realiza de forma exclusiva mediante su CLI[26] y la edición de manifiestos

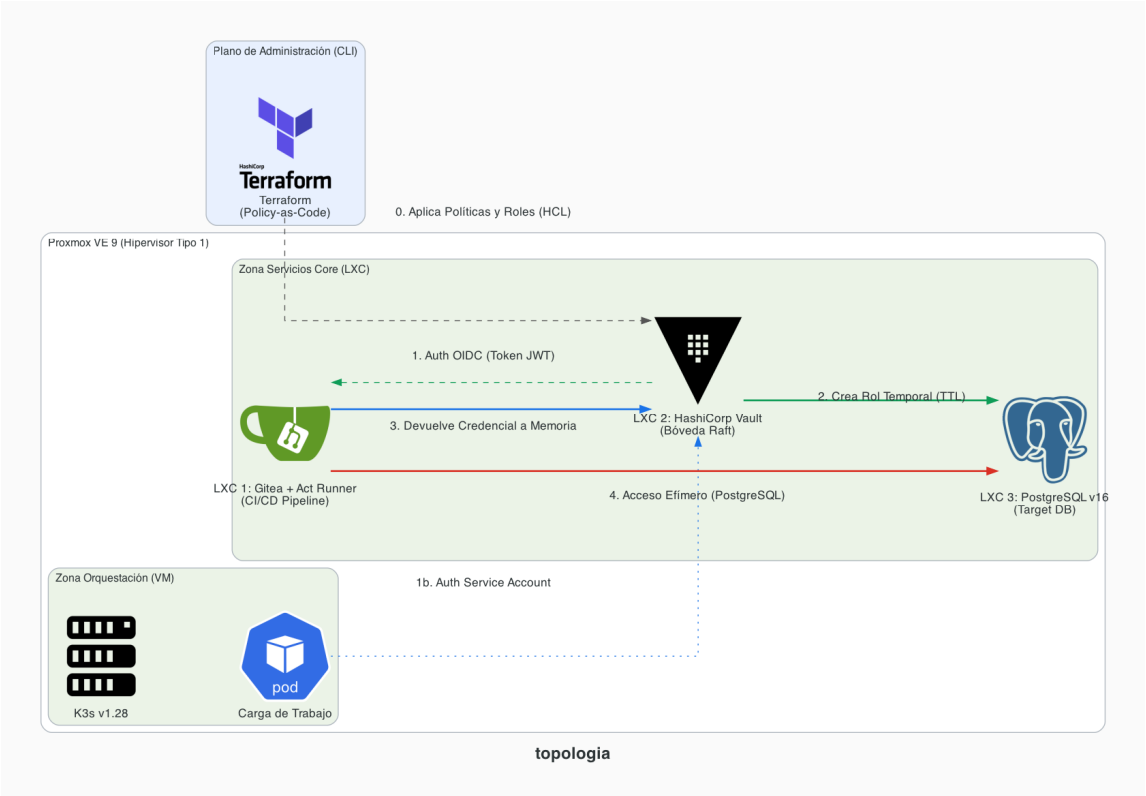

Figura 2. Arquitectura DevSecOps y flujo de inyección dinámica de credenciales en el laboratorio *on-premise*.

con `vi`. Esto permite una integración natural con repositorios de Infraestructura como Código (Terraform) y mantiene un registro de auditoría estricto, prescindiendo totalmente de interfaces gráficas (HashiCorp, s.f.-a, s.f.-c).

5.2. Estrategia de Despliegue mediante IaC

La orquestación de la seguridad y el control de accesos se define íntegramente a través de archivos de configuración declarativos (HCL) utilizando Terraform. Desde la terminal, mediante la edición de estos manifiestos, se establece la configuración del proveedor de HashiCorp Vault[27], la creación de políticas estructuradas (Policy-as-Code[28]) y la habilitación de los motores de secretos (HashiCorp, s.f.-c).

Esta estrategia permite versionar la infraestructura de seguridad directamente en el repositorio de Gitea. Al aplicar los cambios de estado de forma determinista, se garantiza la inmutabilidad del entorno y se elimina la configuración manual. Al prescindir de interfaces gráficas y operar exclusivamente mediante CLI[29], cualquier modificación en los roles o en los tiempos de vida (TTL) de los secretos queda registrada en el control de versiones, habilitando un proceso de revisión por pares (Pull Requests) antes de su aplicación en producción (HashiCorp, s.f.-c).

5.2.1. Instalación de dependencias en Proxmox Para garantizar la reproducibilidad del entorno de laboratorio en el hipervisor Proxmox, la fase inicial del despliegue requiere la preparación del host. Este proceso se ha automatizado mediante un script de inicialización (*shell script*) que estandariza las dependencias operativas antes de la ejecución de cualquier manifiesto declarativo.

El script cumple tres funciones críticas en la cadena de aprovisionamiento:

- **Gestión de Repositorios y Paquetes:** Se integran las claves GPG oficiales de HashiCorp en el anillo de claves del sistema para validar la autenticidad de los binarios, procediendo a la instalación de Terraform y Ansible. La presencia de estas herramientas en el nodo de control es un prerrequisito para ejecutar la orquestación.
- **Obtención de Imágenes Cloud-Init:** Para el despliegue de la máquina virtual destinada al clúster de k3s[30], se requiere una imagen base preparada para la inyección de metadatos. El script descarga la imagen genérica oficial de Debian 12 (`qcow2`) y la aprovisiona en el almacenamiento local del hipervisor (`/var/lib/vz/template/iso`).
- **Estandarización del Formato:** La imagen descargada se renombra automáticamente con la extensión `.img`, un requisito del proveedor de Terraform para Proxmox al procesar plantillas pre-configuradas de Cloud-Init.

Una vez finalizada la ejecución de este script, el host expone las capacidades necesarias para interpretar los manifiestos HCL y aplicar las configuraciones post-despliegue, cerrando la brecha entre la infraestructura vacía y el inicio del ciclo de vida definido por software.

```bash
#!/bin/bash
set -euo pipefail

echo "[*] 1. Instalando dependencias base y repositorios..."
apt-get update && apt-get install -y gnupg software-properties-common curl
    wget

# Añadir repositorio de HashiCorp
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | tee
    /usr/share/keyrings/hashicorp-archive-keyring.gpg >/dev/null
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" | tee
    /etc/apt/sources.list.d/hashicorp.list

echo "[*] 2. Instalando Terraform y Ansible..."
apt-get update
apt-get install -y terraform ansible

echo "[*] 3. Descargando imagen Debian 12 Cloud-Init a local:iso..."
ISO_DIR="/var/lib/vz/template/iso"
DEBIAN_IMG="debian-12-genericcloud-amd64.img"
# URL oficial de la imagen genérica de Cloud para Debian 12
URL="https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-genericcloud-amd64.qcow2"

mkdir -p "$ISO_DIR"
if [ ! -w "$ISO_DIR" ]; then
    echo "[ERROR] No se puede escribir en $ISO_DIR. Ejecuta el script con
        permisos adecuados."
    exit 1
fi

if [ ! -f "$ISO_DIR/$DEBIAN_IMG" ]; then
    # Descargamos como qcow2 y lo renombramos a .img para el Terraform
    wget -O "$ISO_DIR/$DEBIAN_IMG" "$URL"
    if [ ! -s "$ISO_DIR/$DEBIAN_IMG" ]; then
        echo "[ERROR] La descarga de $DEBIAN_IMG ha fallado o está vacía."
        exit 1
    fi
    echo "[+] Imagen descargada correctamente en $ISO_DIR/$DEBIAN_IMG"
else
```

```
    echo "[!] La imagen ya existe, saltando descarga."
fi

echo "[*] 4. Verificando instalaciones..."
terraform -v
ansible --version
ls -lh "$ISO_DIR/$DEBIAN_IMG"

echo -e "\n[OK] Entorno listo."
```

Listing 1.1. Instalación de dependencias en Proxmox

5.2.2. Aprovisionamiento Declarativo con Terraform Con el host preparado tras la ejecución del script de inicialización, la instanciación de la infraestructura se lleva a cabo mediante el archivo `main.tf`. Este manifiesto utiliza el proveedor `bpg/proxmox` para interactuar con la API del hipervisor. Dado que se opera en un entorno local, se define el parámetro `insecure = true` para permitir la comunicación sin requerir una CA interna.

El punto crítico para mantener la automatización ininterrumpida es la directiva `user_account`, la cual inyecta la variable `var.ssh_public_key` en cada nodo. Esta acción garantiza que, desde el primer segundo de vida del sistema operativo, el entorno cuente con acceso por clave asimétrica. Esto erradica el uso de credenciales por defecto y deja la infraestructura inmediatamente accesible para la posterior configuración de los servicios.

```
terraform {
  required_providers {
    proxmox = {
      source  = "bpg/proxmox"
      version = "~> 0.46.1"
    }
    vault = {
      source  = "hashicorp/vault"
      version = "~> 4.5"
    }
  }
}

provider "proxmox" {
  endpoint = var.proxmox_endpoint
  username = var.proxmox_username
  password = var.proxmox_password
```

```
  insecure = var.proxmox_insecure # Solo para entornos de laboratorio
      locales sin CA interna
}

provider "vault" {
  address = var.vault_address
  token   = var.vault_token
}

# ==========================================
# LXC 1: Gitea + Act Runner
# ==========================================
resource "proxmox_virtual_environment_container" "lxc_gitea" {
  node_name = var.proxmox_node
  vm_id     = var.vm_ids.gitea

  initialization {
    hostname = "gitea-runner"
    ip_config {
      ipv4 {
        address = var.lab_ipv4.gitea
        gateway = var.lab_ipv4.gateway
      }
    }
    user_account {
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
    size         = 20
  }

  cpu { cores = 2 }
  memory { dedicated = 2048 }

  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
```

```
  }
}

# ==========================================
# LXC 2: HashiCorp Vault
# ==========================================
resource "proxmox_virtual_environment_container" "lxc_vault" {
  node_name = var.proxmox_node
  vm_id     = var.vm_ids.vault

  initialization {
    hostname = "vault-server"
    ip_config {
      ipv4 {
        address = var.lab_ipv4.vault
        gateway = var.lab_ipv4.gateway
      }
    }
    user_account {
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
    size         = 10
  }

  cpu { cores = 2 }
  memory { dedicated = 2048 }

  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
  }
}

# ==========================================
# LXC 3: PostgreSQL
# ==========================================
```

```
resource "proxmox_virtual_environment_container" "lxc_postgres" {
  node_name = var.proxmox_node
  vm_id     = var.vm_ids.postgres

  initialization {
    hostname = "postgres-db"
    ip_config {
      ipv4 {
        address = var.lab_ipv4.postgres
        gateway = var.lab_ipv4.gateway
      }
    }
    user_account {
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
    size         = 20
  }

  cpu { cores = 2 }
  memory { dedicated = 4096 }

  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
  }
}

# ==========================================
# VM 1: K3s (Kubernetes)
# ==========================================
resource "proxmox_virtual_environment_vm" "vm_k3s" {
  node_name = var.proxmox_node
  vm_id     = var.vm_ids.k3s
  name      = "k3s-cluster"

  agent {
```

```
    enabled = true
  }

  cpu {
    cores = 4
    type  = "x86-64-v2-AES"
  }

  memory {
    dedicated = 8192
  }

  disk {
    datastore_id = "nvme-data"
    file_format  = "raw"
    interface    = "scsi0"
    size         = 40
  }

  initialization {
    ip_config {
      ipv4 {
        address = var.lab_ipv4.k3s
        gateway = var.lab_ipv4.gateway
      }
    }
    user_account {
      username = "debian"
      keys     = [var.ssh_public_key]
    }
  }

  # Requiere una imagen Cloud-Init precargada en Proxmox
  # file_id = "local:iso/debian-12-genericcloud-amd64.img"
}
```

Listing 1.2. Aprovisionamiento de infraestructura lógica mediante Terraform

5.2.3. Configuración post-despliegue con Ansible Una vez aprovisionados los nodos base mediante Terraform, la configuración de servicios se ejecuta con Ansible para mantener separación

de responsabilidades entre infraestructura y estado de aplicación. El playbook orquestador define una secuencia idempotente por rol técnico (Gitea Runner, Vault, PostgreSQL y K3s), permitiendo repetir el proceso sin introducir deriva en configuraciones ya convergidas.

```
---
- import_playbook: lxc_gitea.yml
- import_playbook: lxc_vault.yml
- import_playbook: lxc_postgres.yml
- import_playbook: vm_k3s.yml
```

Listing 1.3. Orquestación de configuración por nodo mediante Ansible

En particular, el nodo de Vault[31] se configura con un playbook dedicado que instala el binario desde repositorio oficial, define `vault.hcl`, habilita almacenamiento local tipo Raft[32] y asegura la activación del servicio. Esta separación favorece trazabilidad operativa y revisión por pares de cambios de hardening y disponibilidad.

```
---
- name: Configure LXC Vault node
  hosts: vault_servers
  become: true
  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install base packages
      ansible.builtin.apt:
        name: "{{ common_packages + ['jq', 'unzip'] }}"
        state: present

    - name: Add HashiCorp apt keyring
      ansible.builtin.shell: |
        set -euo pipefail
        wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor
            >/usr/share/keyrings/hashicorp-archive-keyring.gpg
      args:
        creates: /usr/share/keyrings/hashicorp-archive-keyring.gpg

    - name: Add HashiCorp apt repository
      ansible.builtin.copy:
```

```
    dest: /etc/apt/sources.list.d/hashicorp.list
    content: "deb
        [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
        https://apt.releases.hashicorp.com bookworm main\n"
    mode: "0644"

- name: Install Vault package
  ansible.builtin.apt:
    update_cache: true
    name: "vault={{ vault_version }}*"
    state: present

- name: Create Vault configuration directory
  ansible.builtin.file:
    path: /etc/vault.d
    state: directory
    owner: vault
    group: vault
    mode: "0750"

- name: Configure Vault server
  ansible.builtin.copy:
    dest: /etc/vault.d/vault.hcl
    owner: vault
    group: vault
    mode: "0640"
    content: |
      ui = true
      disable_mlock = true
      api_addr = "{{ vault_api_addr }}"
      cluster_addr = "http://192.168.1.220:8201"
      cluster_name = "{{ vault_cluster_name }}"
      storage "raft" {
        path = "/opt/vault/data"
        node_id = "vault-1"
      }
      listener "tcp" {
        address     = "0.0.0.0:8200"
        tls_disable = 1
      }
```

```
- name: Ensure Vault data directory exists
  ansible.builtin.file:
    path: /opt/vault/data
    state: directory
    owner: vault
    group: vault
    mode: "0750"

- name: Install container-compatible Vault systemd unit
  ansible.builtin.copy:
    dest: /etc/systemd/system/vault.service
    mode: "0644"
    content: |
      [Unit]
      Description=HashiCorp Vault
      After=network-online.target
      Wants=network-online.target

      [Service]
      User=vault
      Group=vault
      ExecStart=/usr/bin/vault server -config=/etc/vault.d/vault.hcl
      ExecReload=/bin/kill --signal HUP $MAINPID
      KillMode=process
      KillSignal=SIGINT
      Restart=on-failure
      RestartSec=5
      LimitNOFILE=65536

      [Install]
      WantedBy=multi-user.target

- name: Reload systemd for Vault unit override
  ansible.builtin.systemd:
    daemon_reload: true

- name: Enable and start Vault
  ansible.builtin.systemd:
    name: vault
```

```
      enabled: true
      state: started

  - name: Create marker for vault node
    ansible.builtin.copy:
      dest: /etc/tfm-node-role
      content: "vault-server\n"
      mode: "0644"
```

Listing 1.4. Configuración del nodo Vault mediante Ansible

5.2.4. Anexo de código fuente del laboratorio Para garantizar trazabilidad completa del entregable técnico, se incorpora a continuación el conjunto de ficheros de automatización que componen el despliegue y la configuración del laboratorio (excluyendo exclusivamente scripts de generación de diagramas).

Guía de lectura del anexo El anexo se organiza siguiendo el mismo flujo operativo del laboratorio:

1. **Definición de infraestructura (Terraform):** primero se declaran variables de entrada, recursos y salidas para trazabilidad.
2. **Configuración de secretos dinámicos (Vault):** se habilita el *database secrets engine*, la conexión a PostgreSQL y el rol de emisión efímera.
3. **Convergencia de servicios (Ansible):** se aplica configuración post-despliegue por rol de nodo (runner, vault, base de datos y clúster).
4. **Automatización de inventario:** se deriva inventario Ansible a partir de `terraform output` para reducir errores manuales.

Criterios técnicos documentados La inclusión de estos ficheros no persigue únicamente exhaustividad, sino demostrar propiedades verificables del diseño:

- **Idempotencia:** los playbooks de Ansible están estructurados para re-ejecución segura sin efectos colaterales sobre estados convergidos.
- **Separación de responsabilidades:** Terraform gestiona estado de infraestructura; Ansible gestiona estado de servicio y hardening básico.
- **Trazabilidad de cambios:** variables, outputs, políticas y scripts quedan versionados y auditables en repositorio.
- **Control de secretos:** credenciales sensibles se parametrizan (no embebidas en código), y la emisión de acceso a datos se modela con TTL y revocación.

Cobertura funcional del código incluido Los listados de Terraform (`variables.tf`, `main.tf`, `outputs.tf` y `vault_bootstrap.tf`) cubren aprovisionamiento de cómputo, red, autenticación de proveedor y bootstrap de secretos dinámicos. Los listados de Ansible (`ansible.cfg`, inventario, `group_vars` y playbooks por nodo) cubren instalación de dependencias, activación de servicios, configuración mínima de Vault/K3s/PostgreSQL y etiquetado funcional del nodo para validación posterior.

```
variable "proxmox_endpoint" {
  type        = string
  description = "URL de la API de Proxmox"
  default     = "https://10.0.0.10:8006/"
}

variable "proxmox_insecure" {
  type        = bool
  description = "Permite omitir la validacion TLS del endpoint (true en
      laboratorio local)."
  default     = true
}

variable "proxmox_node" {
  type        = string
  description = "Nombre del nodo de Proxmox"
  default     = "pve"
}

variable "proxmox_username" {
  type        = string
  description = "Usuario para autenticacion contra la API de Proxmox (ej:
      user@pve)."
  default     = ""
}

variable "proxmox_password" {
  type        = string
  description = "Contrasena para autenticacion de API en Proxmox. Definir
      via tfvars/entorno."
  default     = ""
  sensitive   = true
}

variable "ssh_public_key" {
```

```
  type        = string
  description = "Clave publica SSH para inyectar en las maquinas"

  validation {
    condition     = length(trimspace(var.ssh_public_key)) > 0
    error_message = "ssh_public_key no puede estar vacia."
  }
}

variable "lxc_template_id" {
  type        = string
  description = "ID del template LXC de Debian/Ubuntu"
  default     = "local:vztmpl/debian-13-standard_13.1-2_amd64.tar.zst"
}

variable "vm_ids" {
  description = "IDs de recursos en Proxmox para el laboratorio."
  type = object({
    gitea    = number
    vault    = number
    postgres = number
    k3s      = number
  })
  default = {
    gitea    = 501
    vault    = 502
    postgres = 503
    k3s      = 504
  }
}

variable "lab_ipv4" {
  description = "Direccionamiento IPv4 del laboratorio (con CIDR)."
  type = object({
    gitea    = string
    vault    = string
    postgres = string
    k3s      = string
    gateway  = string
  })
```

```
  default = {
    gitea    = "192.168.1.211/24"
    vault    = "192.168.1.220/24"
    postgres = "192.168.1.230/24"
    k3s      = "192.168.1.240/24"
    gateway  = "192.168.1.1"
  }
}

variable "enable_vault_bootstrap" {
  type        = bool
  description = "Habilita la configuracion de Vault (policy + database
      secrets engine) desde Terraform."
  default     = false
}

variable "vault_address" {
  type        = string
  description = "URL de Vault (ej: http://10.0.0.20:8200)."
  default     = "http://192.168.1.220:8200"
}

variable "vault_token" {
  type        = string
  description = "Token de Vault con permisos para configurar
      auth/policies/secrets engines."
  default     = ""
  sensitive   = true
}

variable "vault_db_admin_username" {
  type        = string
  description = "Usuario administrador de PostgreSQL usado por Vault para
      crear roles dinamicos."
  default     = "postgres"
}

variable "vault_db_admin_password" {
  type        = string
```

```
  description = "Contrasena del usuario administrador de PostgreSQL para
      Vault database secrets engine."
  default     = ""
  sensitive   = true
}
```

Listing 1.5. Variables de entrada para Terraform

```
output "lxc_gitea_ip" {
  value =
      proxmox_virtual_environment_container.lxc_gitea.initialization[0].ip_config[0].ipv4
}

output "lxc_gitea_info" {
  value = {
    node_name = proxmox_virtual_environment_container.lxc_gitea.node_name
    vm_id     = proxmox_virtual_environment_container.lxc_gitea.vm_id
    hostname  =
        proxmox_virtual_environment_container.lxc_gitea.initialization[0].hostname
  }
}

output "lxc_vault_ip" {
  value =
      proxmox_virtual_environment_container.lxc_vault.initialization[0].ip_config[0].ipv4
}

output "lxc_vault_info" {
  value = {
    node_name = proxmox_virtual_environment_container.lxc_vault.node_name
    vm_id     = proxmox_virtual_environment_container.lxc_vault.vm_id
    hostname  =
        proxmox_virtual_environment_container.lxc_vault.initialization[0].hostname
  }
}

output "lxc_postgres_ip" {
  value =
      proxmox_virtual_environment_container.lxc_postgres.initialization[0].ip_config[0].ip
}
```

```
output "lxc_postgres_info" {
  value = {
    node_name =
        proxmox_virtual_environment_container.lxc_postgres.node_name
    vm_id     = proxmox_virtual_environment_container.lxc_postgres.vm_id
    hostname  =
        proxmox_virtual_environment_container.lxc_postgres.initialization[0].hostname
  }
}

output "vm_k3s_ip" {
  value =
      proxmox_virtual_environment_vm.vm_k3s.initialization[0].ip_config[0].ipv4[0].address
}

output "vm_k3s_info" {
  value = {
    node_name = proxmox_virtual_environment_vm.vm_k3s.node_name
    vm_id     = proxmox_virtual_environment_vm.vm_k3s.vm_id
    name      = proxmox_virtual_environment_vm.vm_k3s.name
  }
}

output "vault_bootstrap_enabled" {
  value = var.enable_vault_bootstrap
}

output "vault_db_role_name" {
  value = var.enable_vault_bootstrap ?
      vault_database_secret_backend_role.readonly[0].name : null
}
```

Listing 1.6. Outputs de Terraform para trazabilidad de recursos

```
locals {
  postgres_host = split("/", var.lab_ipv4.postgres)[0]
}

resource "vault_mount" "database" {
  count = var.enable_vault_bootstrap ? 1 : 0
```

```
  path        = "database"
  type        = "database"
  description = "Database secrets engine para emision dinamica de
      credenciales PostgreSQL"
}

resource "vault_database_secret_backend_connection" "postgres" {
  count = var.enable_vault_bootstrap ? 1 : 0

  backend       = vault_mount.database[0].path
  name          = "postgres"
  allowed_roles = ["readonly-role"]

  postgresql {
    connection_url =
        "postgresql://{{username}}:{{password}}@${local.postgres_host}:5432/postgres?sslmo
    username       = var.vault_db_admin_username
    password       = var.vault_db_admin_password
  }
}

resource "vault_database_secret_backend_role" "readonly" {
  count = var.enable_vault_bootstrap ? 1 : 0

  backend = vault_mount.database[0].path
  name    = "readonly-role"
  db_name = vault_database_secret_backend_connection.postgres[0].name

  creation_statements = [
    "CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID
        UNTIL '{{expiration}}';",
    "GRANT CONNECT ON DATABASE postgres TO \"{{name}}\";",
    "GRANT USAGE ON SCHEMA public TO \"{{name}}\";",
    "GRANT SELECT ON ALL TABLES IN SCHEMA public TO \"{{name}}\";"
  ]

  revocation_statements = [
    "REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM
        \"{{name}}\";",
    "REVOKE USAGE ON SCHEMA public FROM \"{{name}}\";",
```

```
    "REVOKE CONNECT ON DATABASE postgres FROM \"{{name}}\";",
    "DROP ROLE IF EXISTS \"{{name}}\";"
  ]

  default_ttl = 300
  max_ttl     = 900
}

resource "vault_policy" "ci_dynamic_db_read" {
  count = var.enable_vault_bootstrap ? 1 : 0

  name = "ci-dynamic-db-read"

  policy = <<EOT
path "database/creds/readonly-role" {
  capabilities = ["read"]
}
EOT
}
```

Listing 1.7. Bootstrap de Vault y motor dinámico de base de datos en Terraform

```
[defaults]
inventory = inventories/lab/hosts.yml
host_key_checking = False
timeout = 20
retry_files_enabled = False
stdout_callback = default
result_format = yaml
interpreter_python = auto_silent

[privilege_escalation]
become = True
become_method = sudo
become_ask_pass = False
```

Listing 1.8. Configuración global de Ansible

```
all:
  vars:
    ansible_ssh_private_key_file: ~/.ssh/tfm_proxmox_temp
```

```
  children:
    gitea_runner:
      hosts:
        gitea:
          ansible_host: 192.168.1.211
          ansible_user: root
    vault_servers:
      hosts:
        vault:
          ansible_host: 192.168.1.220
          ansible_user: root
    postgres_servers:
      hosts:
        postgres:
          ansible_host: 192.168.1.230
          ansible_user: root
    k3s_servers:
      hosts:
        k3s:
          ansible_host: 192.168.1.240
          ansible_user: debian
```

Listing 1.9. Inventario del laboratorio en Ansible

```
---
common_packages:
  - curl
  - ca-certificates
  - gnupg
  - apt-transport-https
  - python3
  - python3-pip

vault_version: "1.17.5"
vault_cluster_name: "vault-lab"
vault_api_addr: "http://192.168.1.220:8200"

postgresql_version: "17"
postgresql_db_name: "postgres"
postgresql_admin_user: "postgres"
```

```
k3s_install_channel: "stable"
```

Listing 1.10. Variables compartidas de Ansible para todos los nodos

```
---
- name: Configure LXC Gitea runner node
  hosts: gitea_runner
  become: true
  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install base packages
      ansible.builtin.apt:
        name: "{{ common_packages + ['git', 'jq'] }}"
        state: present

    - name: Ensure gitea service account exists
      ansible.builtin.user:
        name: gitea
        shell: /bin/bash
        create_home: true
        state: present

    - name: Create Gitea required directories
      ansible.builtin.file:
        path: "{{ item.path }}"
        state: directory
        owner: "{{ item.owner }}"
        group: "{{ item.group }}"
        mode: "{{ item.mode }}"
      loop:
        - { path: /var/lib/gitea, owner: gitea, group: gitea, mode: "0750"
            }
        - { path: /var/lib/gitea/custom, owner: gitea, group: gitea, mode:
            "0750" }
        - { path: /var/lib/gitea/data, owner: gitea, group: gitea, mode:
            "0750" }
```

```
      - { path: /var/lib/gitea/log, owner: gitea, group: gitea, mode:
          "0750" }
      - { path: /etc/gitea, owner: gitea, group: gitea, mode: "0770" }

  - name: Install Gitea binary
    ansible.builtin.shell: |
      set -euo pipefail
      curl -fsSL
          "https://dl.gitea.com/gitea/1.22.6/gitea-1.22.6-linux-amd64" -o
          /usr/local/bin/gitea
      chmod 0755 /usr/local/bin/gitea
    args:
      creates: /usr/local/bin/gitea

  - name: Create basic Gitea app configuration
    ansible.builtin.copy:
      dest: /etc/gitea/app.ini
      owner: gitea
      group: gitea
      mode: "0660"
      content: |
        APP_NAME = TFM Gitea
        RUN_USER = gitea
        RUN_MODE = prod
        [server]
        DOMAIN = 192.168.1.211
        HTTP_PORT = 3000
        ROOT_URL = http://192.168.1.211:3000/
        [database]
        DB_TYPE = sqlite3
        PATH = /var/lib/gitea/data/gitea.db
        [security]
        INSTALL_LOCK = true
        SECRET_KEY = tfm-lab-secret-key

  - name: Install Gitea systemd unit
    ansible.builtin.copy:
      dest: /etc/systemd/system/gitea.service
      mode: "0644"
      content: |
```

```
        [Unit]
        Description=Gitea (Git with a cup of tea)
        After=network.target

        [Service]
        Type=simple
        User=gitea
        Group=gitea
        WorkingDirectory=/var/lib/gitea/
        ExecStart=/usr/local/bin/gitea web --config /etc/gitea/app.ini
        Restart=always
        Environment=USER=gitea HOME=/home/gitea
            GITEA_WORK_DIR=/var/lib/gitea

        [Install]
        WantedBy=multi-user.target

    - name: Reload systemd
      ansible.builtin.systemd:
        daemon_reload: true

    - name: Ensure Gitea service is running
      ansible.builtin.systemd:
        name: gitea
        enabled: true
        state: started

    - name: Create marker for runner node
      ansible.builtin.copy:
        dest: /etc/tfm-node-role
        content: "gitea-runner\n"
        mode: "0644"
```

Listing 1.11. Playbook de configuración para nodo Gitea Runner

```
---
- name: Configure LXC PostgreSQL node
  hosts: postgres_servers
  become: true
  tasks:
    - name: Update apt cache
```

```
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install PostgreSQL server
      ansible.builtin.apt:
        name:
          - postgresql
          - postgresql-client
          - python3-psycopg2
        state: present

    - name: Ensure PostgreSQL is running
      ansible.builtin.systemd:
        name: postgresql
        enabled: true
        state: started

    - name: Create marker for postgres node
      ansible.builtin.copy:
        dest: /etc/tfm-node-role
        content: "postgres-db\n"
        mode: "0644"
```

Listing 1.12. Playbook de configuración para nodo PostgreSQL

```
---
- name: Configure VM K3s node
  hosts: k3s_servers
  become: true
  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Install base packages
      ansible.builtin.apt:
        name: "{{ common_packages + ['curl', 'bash'] }}"
        state: present
```

```
    - name: Install K3s server if missing
      ansible.builtin.shell: |
        set -euo pipefail
        curl -sfL https://get.k3s.io | INSTALL_K3S_CHANNEL={{
            k3s_install_channel }} sh -
      args:
        executable: /bin/bash
        creates: /usr/local/bin/k3s

    - name: Ensure k3s service is running
      ansible.builtin.systemd:
        name: k3s
        enabled: true
        state: started

    - name: Create marker for k3s node
      ansible.builtin.copy:
        dest: /etc/tfm-node-role
        content: "k3s-cluster\n"
        mode: "0644"
```

Listing 1.13. Playbook de configuración para nodo K3s

```
#!/usr/bin/env bash
set -euo pipefail

TF_DIR="${1:-../terraform}"
OUT_FILE="${2:-inventories/lab/hosts.yml}"
SSH_KEY="${3:-~/.ssh/tfm_proxmox_temp}"

cd "$(dirname "$0")/.."

tf_output() {
  terraform -chdir="$TF_DIR" output -raw "$1" 2>/dev/null || true
}

gitea_ip="$(tf_output lxc_gitea_ip)"
vault_ip="$(tf_output lxc_vault_ip)"
postgres_ip="$(tf_output lxc_postgres_ip)"
k3s_ip="$(tf_output vm_k3s_ip)"
```

```
cat > "$OUT_FILE" <<EOF
all:
  vars:
    ansible_ssh_private_key_file: $SSH_KEY
  children:
    gitea_runner:
      hosts:
        gitea:
          ansible_host: ${gitea_ip:-10.0.0.11/24}
          ansible_user: root
    vault_servers:
      hosts:
        vault:
          ansible_host: ${vault_ip:-10.0.0.20/24}
          ansible_user: root
    postgres_servers:
      hosts:
        postgres:
          ansible_host: ${postgres_ip:-10.0.0.30/24}
          ansible_user: root
    k3s_servers:
      hosts:
        k3s:
          ansible_host: ${k3s_ip:-10.0.0.40/24}
          ansible_user: debian
EOF

# Remove CIDR suffix if present.
sed -i.bak 's#/24##g' "$OUT_FILE" && rm -f "$OUT_FILE.bak"
echo "[OK] Inventory generated at: $OUT_FILE"
```

Listing 1.14. Script de generación de inventario Ansible a partir de outputs Terraform

Generación Dinámica de Credenciales en PostgreSQL

Tras la autenticación exitosa del pipeline de CI/CD mediante OIDC[33], la carga de trabajo requiere acceso al motor de base de datos relacional ubicado en el LXC[34] 3. En lugar de inyectar una cadena de conexión estática, el runner realiza una petición a la API de Vault[35], configurado previamente con el motor de base de datos (Database Secrets Engine), interactúa de forma nativa con PostgreSQL para crear un rol de usuario efímero con permisos limitados (ej. `SELECT` sobre

tablas específicas). Esta credencial se devuelve al runner en memoria. Una vez finalizado el TTL estipulado, o al concluir la ejecución del trabajo, Vault[36] ejecuta automáticamente una sentencia `DROP ROLE` en la base de datos, revocando el acceso y eliminando cualquier rastro de la credencial (HashiCorp, s.f.-b).

En la configuración de Terraform se detalla la lógica declarativa para aprovisionar el motor de secretos de base de datos en Vault[37]. Este manifiesto establece la conexión con la instancia de PostgreSQL alojada en el LXC[38] 3 y define las sentencias SQL parametrizadas que el sistema ejecutará en tiempo real para generar credenciales de solo lectura (HashiCorp, s.f.-b, s.f.-c).

Al aplicar este manifiesto, la lógica de creación de usuarios queda versionada e inmutable. El comportamiento esperado es que, ante cualquier intento de acceso lateral, la caducidad inherente del usuario (300 segundos) mitigue de forma severa la ventana de exposición del dato.

5.3. Metodología de evaluación

La validación de la arquitectura se basa en criterios funcionales y en evidencia observables en el laboratorio. En concreto, se considera que el diseño funciona si se cumple lo siguiente: (1) la emisión de credenciales para acceso a base de datos es efímera y conduce a la revocación del rol mediante la operación `DROP ROLE`; (2) la trazabilidad de accesos se materializa en eventos de auditoría en formato JSON generados por Vault[39]; y (3) la configuración del entorno y de las políticas se puede reproducir a partir de manifiestos declarativos versionados con Terraform (HashiCorp, s.f.-a, s.f.-b, s.f.-c).

Las fuentes de evidencia utilizadas incluyen la topología del laboratorio (Figura 2), la automatización del host (Listado 1.1), la lógica de aprovisionamiento declarativo (Listado 1.2) y la orquestación de configuración por Ansible (Listados 1.3 y 1.4). Como control de reproducibilidad del código de infraestructura, se incorpora además la validación sintáctica y semántica de manifiestos mediante `terraform validate`, junto con la separación explícita de parámetros sensibles de autenticación de la API de Proxmox respecto al código versionado.

Como verificación operativa complementaria, se ejecutaron pruebas no destructivas por `SSH` sobre el host Proxmox del laboratorio para confirmar identidad de ejecución y estado base de plataforma (`hostname`, `id`, `pve-manager`). Este paso permite acotar el riesgo previo a cualquier *apply*, al validar que el contexto remoto y los permisos de operador son los esperados antes de introducir cambios de estado.

6. Resultados

6.1. Matriz de verificación operativa (evidencia directa)

Con el objetivo de aportar trazabilidad verificable para la defensa, la Tabla 1 resume las comprobaciones ejecutadas sobre el entorno real de Proxmox y su estado observado en esta iteración.

Objetivo evaluado	Prueba ejecutada	Resultado observado
Existencia y arranque de nodos del laboratorio (501–504)	Inspección de estado con `pct list` y `qm list/status` en Proxmox	Nodos creados y en ejecución: `gitea-runner` (501), `vault-server` (502), `postgres-db` (503), `k3s-cluster` (504).
Coherencia de topología y direccionamiento	Revisión de configuración con `pct config` y `qm config`	Direccionamiento aplicado en red de laboratorio (`192.168.1.0/24`) y mapeo funcional alineado con la Figura 2.
Accesibilidad de automatización sobre nodos de servicio	Validación de conectividad Ansible (`ansible -m ping`) desde el host Proxmox hacia 501–503	Respuesta `pong` en los tres contenedores de servicio, confirmando canal de automatización operativo para la fase de configuración.
Convergencia funcional de servicios de aplicación	Ejecución de playbooks Ansible por rol (Gitea, Vault y PostgreSQL) y comprobación con `systemctl is-active` + puertos en escucha	Servicios convergidos y activos: Gitea (`:3000`), Vault (`:8200`) y PostgreSQL (`:5432`) en sus nodos correspondientes.
Estado operacional de la bóveda de secretos (Vault[40])	Inicialización y desellado controlado mediante `vault operator init/unseal` + comprobación de `/v1/sys/health`	Vault[41] queda en estado `initialized=true`, `sealed=false`, modo `active`, con *health check* HTTP 200 y almacenamiento Raft[42] operativo.
Estado operacional del clúster k3s[43] (VM 504)	Comprobación de red, `SSH`, `qemu-guest-agent`, servicio `k3s` y `kubectl get nodes -o wide`	Nodo `k3s-cluster` accesible y en estado `Ready`; servicios `ssh`, `qemu-guest-agent` y `k3s` activos en la VM de orquestación.

Cuadro 1. Matriz de verificación operativa del laboratorio: objetivo, prueba ejecutada y resultado observado.

6.2. Emisión dinámica de credenciales (TTL) y revocación

Durante la ejecución del flujo propuesto, el runner obtiene credenciales de forma efímera a partir de Vault[44], asociadas a un TTL[45] configurado para limitar el periodo de exposición del secreto. La evidencia funcional se sustenta en que el aprovisionamiento de la lógica de emisión y revocación se define mediante manifiestos Terraform (Listado 1.2) que conectan Vault[46] con PostgreSQL y gobiernan el comportamiento de acceso. En consecuencia, al finalizar el trabajo (y/o expirar el periodo asociado al TTL[47]), el acceso se revoca mediante `DROP ROLE` en PostgreSQL, eliminando el rol efímero y reduciendo el impacto temporal de una posible exposición (HashiCorp, s.f.-b).

Como validación operativa directa en Proxmox, se ejecutó una prueba extremo a extremo sobre el motor dinámico de base de datos de Vault[48] con una credencial efímera real:

1. Se solicitó una credencial dinámica (`database/creds/readonly-role`) y se obtuvo un usuario temporal único con *lease* asociado.
2. Con esa credencial se ejecutó una consulta satisfactoria en PostgreSQL (`pre_revoke_query=ok`).
3. Se revocó la *lease* en modo síncrono (`revoke_rc=0` y confirmación de revocación exitosa).
4. Tras la revocación, la misma credencial dejó de autenticar (`post_revoke_query=failed_expected`) y el rol temporal ya no existía en `pg_roles` (`post_revoke_role_exists=no`).

Este resultado confirma de forma empírica el comportamiento esperado de emisión efímera y revocación efectiva, alineado con la hipótesis de reducción de persistencia del riesgo por Secret Sprawl[49].

El entorno que soporta esta secuencia está descrito en la Figura 2.

6.3. Trazabilidad de auditoría y detección

La segunda dimensión evaluada es la trazabilidad. El motor de auditoría de Vault[50] registra cada interacción en formato JSON con campos consistentes (identidad solicitante, política evaluada, ruta del secreto y timestamp). Esta evidencia se alinea con la arquitectura del laboratorio descrita en la Figura 2 y facilita la correlación de eventos hacia un sistema de monitorización tipo SIEM[51], habilitando la detección de anomalías asociadas a intentos fallidos o patrones de acceso no esperados (HashiCorp, s.f.-a; National Institute of Standards and Technology, 2020).

6.4. Reproducibilidad del despliegue mediante IaC

La reproducibilidad del despliegue se valida verificando que el laboratorio on-premise se construye desde código versionado y declarativo. El aprovisionamiento del host se automatiza mediante el script del Listado 1.1, mientras que la configuración de políticas, motores y roles se define a través de manifiestos Terraform (Listado 1.2). Sobre esta base, la configuración específica de cada nodo (Gitea Runner, Vault, PostgreSQL y K3s) se orquesta mediante playbooks Ansible idempotentes (Listados 1.3 y 1.4), separados por rol técnico y reutilizables en nuevas iteraciones del

laboratorio. Este enfoque reduce la deriva operativa entre ejecuciones y refuerza la hipótesis de una implementación inmutable y auditable en el SDLC[52] (HashiCorp, s.f.-c).

En la validación experimental de esta iteración se observó:

1. `terraform validate` finaliza con éxito tras ajustar la validación de entrada de clave pública.
2. El aprovisionamiento controlado en Proxmox genera y arranca los nodos reservados del laboratorio (IDs 501–504) con la asignación funcional esperada (Gitea Runner, Vault, PostgreSQL y K3s).
3. La conectividad de automatización con Ansible es correcta cuando se ejecuta desde el propio host Proxmox hacia los nodos internos (respuesta `pong` en contenedores y acceso `SSH` operativo en la VM k3s[53]).
4. La convergencia de servicios queda validada con evidencias de ejecución (`systemctl is-active`, versiones de binario y puertos en escucha) en Gitea, Vault y PostgreSQL, incluyendo inicialización y desellado operativo de Vault con respuesta de salud HTTP 200.
5. El clúster k3s[54] queda funcional en la VM 504, con nodo `Ready` validado mediante `kubectl get nodes -o wide`.

6.5. Pruebas complementarias con métricas y casos negativos

Para reforzar la validez experimental, se ejecutó una batería adicional automatizada sobre Proxmox (artefacto: `artifacts/validation/proxmox_validation_results.txt`) con resultados cuantitativos y de control de acceso:

- **Tiempo de emisión de credencial dinámica:** `issue_ms=107` ms.
- **Tiempo de revocación síncrona:** `revoke_ms=103` ms.
- **TTL efectivo de lease:** `lease_ttl_seconds=60`.
- **Prueba positiva de uso:** credencial recién emitida válida en PostgreSQL (`pre_revoke_query=ok`).
- **Prueba negativa de revocación:** tras revocar, la credencial deja de operar (`post_revoke_query=failed_expe`
  y el rol efímero se elimina del catálogo (`post_revoke_role_exists=no`).
- **Prueba negativa de autorización:** un token sin política de lectura a `database/creds/readonly-role` recibe denegación (`unauthorized_token_access=denied_expected`).

Estos resultados incrementan el nivel de evidencia más allá de la validación funcional cualitativa, al demostrar comportamiento temporal y de control de acceso con medidas observables en ejecución real.

7. Discusión

Para sintetizar la relación entre marco regulatorio, controles técnicos y evidencia experimental del trabajo, la Figura 3 presenta la matriz de trazabilidad del TFM.

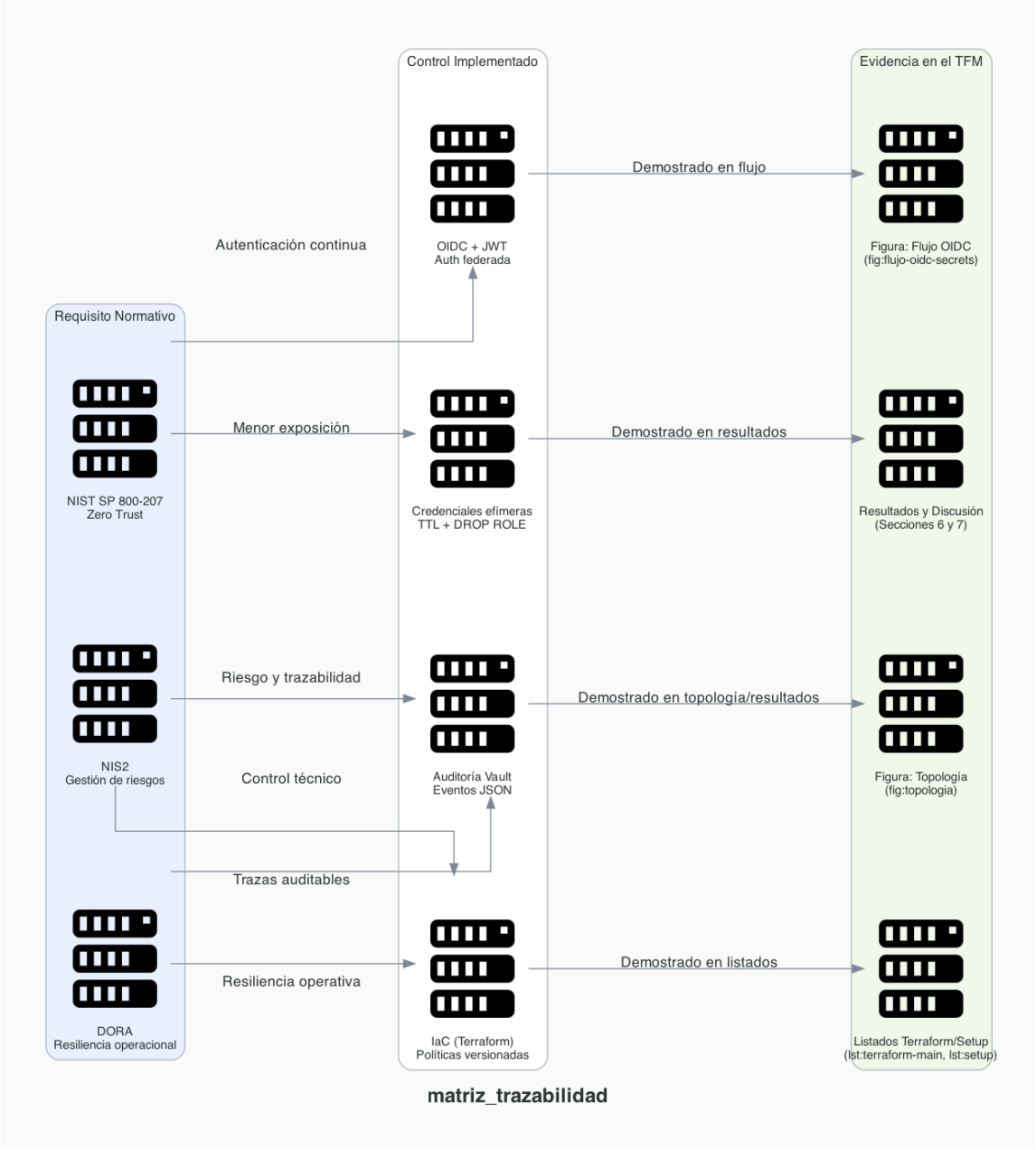

Figura 3. Matriz de trazabilidad entre requisitos normativos (NIST SP 800-207, NIS2 y DORA), controles implementados en la arquitectura y evidencias documentadas en el TFM.

7.1. Confirmación de hipótesis y relación con el Secret Sprawl

Los resultados observados en el laboratorio respaldan la hipótesis principal: la sustitución de credenciales estáticas por emisión dinámica reduce el tiempo durante el cual una credencial podría reutilizarse tras una potencial exposición, mitigando Secret Sprawl[55]. Además, la trazabilidad habilitada en Vault[56] y la arquitectura orientada a prevención son coherentes con buenas prácticas de secret management y secret scanning (GitHub, s.f.; HashiCorp, s.f.-a; OWASP, s.f.).

7.2. Amenazas a la validez y limitaciones

Como limitación, el estudio se basa en un entorno de laboratorio on-premise con componentes específicos (Proxmox, LXC, k3s, Vault[57] y PostgreSQL). La validez externa queda condicionada por la replicabilidad de la configuración y por la calidad de las políticas implementadas. Además, la medición cuantitativa (p. ej., latencias, distribución estadística de tiempos de emisión/revocación o tasa de falsos positivos en detección) no se detalla en esta versión del TFM, por lo que la evaluación se centra en evidencia funcional y trazabilidad operacional.

7.3. Trabajo futuro

Como trabajo futuro se recomienda ampliar el conjunto de pruebas con escenarios de fallo (p. ej., denegaciones por políticas, roturas de validación OIDC, fallos transitorios del motor de auditoría), incorporar métricas cuantitativas del pipeline y realizar correlación real de eventos hacia un sistema de monitorización tipo SIEM[58]. También sería deseable evaluar otras superficies de secretos (certificados, llaves de cifrado, credenciales de servicios) para generalizar el impacto de la arquitectura propuesta (HashiCorp, s.f.-a).

8. Conclusiones

La implementación y evaluación del laboratorio *on-premise* demuestra la viabilidad técnica de erradicar los secretos estáticos en infraestructuras locales con altos requisitos de aislamiento.

La integración de una bóveda criptográfica centralizada, combinada con la federación de identidades OIDC[59], resuelve eficazmente el problema del "secreto cero" sin comprometer la automatización del ciclo CI/CD. Se confirma que abstraer la gestión de acceso a través de herramientas de Infraestructura como Código no solo acota la superficie de exposición ante exfiltraciones, sino que dota al equipo de ingeniería de una operativa ágil y puramente declarativa.

En conclusión, desplazar la seguridad hacia las fases iniciales del desarrollo (*shift-left*) y basar el acceso en validaciones dinámicas de identidad transforma la postura defensiva de la organización, mitigando de forma medible la problemática del *Secret Sprawl* y preparando la infraestructura para futuras adopciones de nubes híbridas.

9. Glosario

Castillo y Foso Modelo de seguridad perimetral tradicional basado en la premisa de defender una red interna segura mediante barreras externas, como firewalls. Asume que los usuarios y dispositivos dentro del perímetro son de confianza, un enfoque que resulta ineficaz en entornos distribuidos modernos.. 43

CI/CD Integración Continua y Despliegue Continuo. Práctica de automatización que integra el desarrollo y el despliegue de software de forma constante y segura.. 43

CLI Command-Line Interface. Interfaz de línea de comandos para operar herramientas y automatizar tareas.. 10, 12, 43

DAST Dynamic Application Security Testing. Pruebas de seguridad sobre una aplicación en ejecución para identificar vulnerabilidades mediante interacciones y análisis dinámico.. 5, 7, 43

HCL HashiCorp Configuration Language. Lenguaje declarativo utilizado para definir la infraestructura en herramientas como Terraform.. 43

IaC Infraestructura como Código. Gestión y aprovisionamiento de la infraestructura a través de código y manifiestos declarativos en lugar de procesos manuales.. 43

IAST Interactive Application Security Testing. Combinación de técnicas de análisis dinámico e instrumentación en tiempo de ejecución para detectar vulnerabilidades con contexto.. 5, 43

JWT JSON Web Token. Estándar abierto (RFC 7519) que define un modo compacto y autónomo para transmitir información de forma segura entre las partes como un objeto JSON.. 43

k3s Distribución ligera de Kubernetes, diseñada para simplificar la instalación y operación en entornos con recursos limitados.. 10, 12, 38, 40, 43

Kubernetes Plataforma de orquestación de contenedores que automatiza despliegue, escalado y operación de aplicaciones.. 10, 43

LXC Linux Containers. Tecnología de contenedores basada en aislamiento a nivel de sistema operativo en Linux.. 10, 36, 37, 43

MFA Multi-Factor Authentication. Autenticación que exige dos o más factores (p. ej., algo que eres, algo que tienes, algo que sabes).. 4, 43

mTLS Mutual Transport Layer Security. Proceso en el que ambas partes de una conexión de red autentican mutuamente sus certificados criptográficos.. 4, 43

OIDC OpenID Connect. Capa de identidad basada en el protocolo OAuth 2.0 que permite a los clientes verificar la identidad de una entidad.. 8, 10, 36, 42, 43

Policy-as-Code Modelo que codifica políticas de seguridad como artefactos versionados y ejecutables automáticamente para aplicar controles consistentes.. 5, 8, 12, 43

Raft Algoritmo de consenso utilizado para replicación y mantenimiento de estado en sistemas distribuidos (p. ej., motores de almacenamiento en Vault).. 10, 19, 38, 43

SAST Static Application Security Testing. Análisis del código fuente en busca de vulnerabilidades aplicando reglas estáticas antes de la ejecución.. 7, 43

SDLC Software Development Life Cycle (Ciclo de Vida del Desarrollo de Software). Proceso integral para el diseño, desarrollo, pruebas y despliegue de software de alta calidad.. 40, 43

Secret Sprawl Dispersión descontrolada de credenciales (claves API, certificados, tokens) a lo largo del ciclo de vida del desarrollo de software.. 39, 42, 43

SIEM Security Information and Event Management. Sistema centralizado que proporciona análisis en tiempo real de alertas de seguridad generadas por el hardware y las aplicaciones de red.. 39, 42, 43

TLS Transport Layer Security. Protocolo criptográfico para proteger comunicaciones en redes.. 4, 43

TTL Time To Live (Tiempo de Vida). Período de validez o caducidad asignado a un recurso, como un token o una credencial efímera, antes de ser revocado automáticamente.. 39, 43

Vault HashiCorp Vault. Bóveda que gestiona secretos y credenciales con cifrado, control de acceso y generación dinámica según políticas.. 6, 8, 10, 12, 19, 36–39, 42, 43

Zero Trust Modelo de ciberseguridad que asume que no existe confianza implícita concedida a los activos o cuentas basándose únicamente en su ubicación física o de red.. 43

10. Bibliografía

Referencias

European Parliament and the Council of the European Union. (2022a). *Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on measures for a high common level of cybersecurity across the Union*. https://eur-lex.europa.eu/eli/dir/2022/2555/oj

European Parliament and the Council of the European Union. (2022b). *Regulation (EU) 2022/2554 of the European Parliament and of the Council of 14 December 2022 on digital operational resilience for the financial sector*. https://eur-lex.europa.eu/eli/reg/2022/2554/oj

GitHub. (s.f.). *About secret scanning*. https://docs.github.com/en/code-security/secret-scanning/about-secret-scanning

HashiCorp. (s.f.-a). *Audit Devices*. https://developer.hashicorp.com/vault/docs/audit

HashiCorp. (s.f.-b). *Database Secrets Engine*. https://developer.hashicorp.com/vault/docs/secrets/databases

HashiCorp. (s.f.-c). *Terraform Documentation*. https://developer.hashicorp.com/terraform/docs

Jones, M. B., & Bradley, J. (2015). *JSON Web Token (JWT)* (Request for Comments N.º 7519). IETF. https://www.rfc-editor.org/rfc/rfc7519

Kubernetes Authors. (s.f.). *Kubernetes Concepts*. https://kubernetes.io/docs/concepts/overview/

National Institute of Standards and Technology. (2020). *Zero Trust Architecture* (Special Publication N.º 800-207). National Institute of Standards and Technology. https://csrc.nist.gov/publications/detail/sp/800-207/final

OpenID Foundation. (2014). *OpenID Connect Core 1.0*. https://openid.net/specs/openid-connect-core-1_0.html

OWASP. (s.f.). *Secrets Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Rancher Labs. (s.f.). *k3s Documentation*. https://docs.k3s.io/